Anexo: Licencias de los componentes de CompanIA

Planteamiento

El robot de CompanIA está formado por varios componentes con dueños y condiciones distintas: el código que desarrollamos nosotros, las librerías y modelos de terceros sobre los que se apoya, los datos con los que se entrena y la documentación. En este anexo definimos, como startup desarrolladora, qué licencia tiene cada uno y por qué.

Para decidir hemos usado cinco criterios, que aplicamos en todos los apartados:

- **Compatibilidad:** la licencia de nuestro código no decide qué piezas podemos usar. Lo que hay que evitar es mezclarlo con código copyleft incompatible o redistribuir lo que no es nuestro.
- **Seguridad física:** el robot convive con personas vulnerables y tendrá que certificarse (Directiva de Máquinas, marcado CE). Ninguna licencia debe obligarnos a permitir modificar el sistema de seguridad de un robot certificado.
- **Privacidad:** nada que contenga o pueda filtrar datos personales se publica, aunque legalmente se pudiera.
- **Transparencia:** lo que decide sobre la seguridad del usuario debe poder auditarse por terceros, en línea con el AI Act.
- **Viabilidad:** somos una startup, así que necesitamos una parte del producto con la que ganar dinero.

El resultado es un modelo de núcleo abierto (open core): abrimos lo que conviene auditar y compartir, y mantenemos cerrado lo que depende de licencias propietarias, contiene datos personales o sostiene el negocio.

Resumen

Componente	Licencia	¿Libre?	Motivo principal
Código propio: motor de decisión, nodos ROS, interfaces	Apache 2.0	Sí	Auditabilidad y compatibilidad con ROS 2
App de familiares y plataforma en la nube	Propietaria	No	Modelo de negocio
Librerías y modelos de terceros	La de cada proveedor	Depende	No la elegimos, la respetamos
Detector de caídas propio	Apache 2.0 (pesos incluidos)	Sí	Transparencia en una función de seguridad
Control y voz afinados (Gemini, Whisper)	Propietaria, con model card pública	No	Licencia de Google y privacidad
Datos de usuarios	Ninguna (RGPD y consentimiento)	No aplica	No son nuestros
Grabaciones de voluntarios	Acceso restringido para investigación	No	Datos personales
Dataset de demostraciones domésticas	CC BY 4.0	Sí (datos abiertos)	Aportación a la comunidad
Documentación	CC BY 4.0	Sí	Máxima reutilización
Diseño del hardware	Propietario	No	Certificación de seguridad

1. Código

1.1 Código propio: Apache 2.0

Aquí entra todo lo que desarrollamos nosotros para unir las piezas: los nodos de ROS que conectan sensores, modelos y motores, el motor de decisión ante riesgos (las reglas explícitas que deciden cómo actuar ante una caída o un cambio de rutina) y las interfaces entre componentes. Es el componente que publicamos como software libre, bajo Apache 2.0 [1].

Las razones son cuatro:

- **Alineación con ROS 2.** Los paquetes estándar de ROS 2, el software que conecta todo el robot, usan Apache 2.0 [4]. Con la misma licencia, nuestro código puede integrarse y contribuirse a ese ecosistema sin fricciones. Esto no significa que todo lo que usamos sea abierto: el modelo de control de Google es propietario, y nuestro código puede llamarlo porque Apache es permisiva, no porque ese modelo sea compatible con Apache.
- **Protege frente a patentes.** Apache 2.0 concede expresamente licencia sobre las patentes de quien contribuye. En robótica, donde hay muchas patentes sobre mecanismos y control, esto da seguridad a quien use o mejore nuestro código. MIT y BSD no incluyen esta cláusula.
- **Hace comprobable lo que prometemos.** Es el argumento ético más fuerte. Desde el primer sprint decimos que el motor de decisión funciona con reglas escritas por personas para que sea predecible y revisable. Si el código está publicado, cualquier auditor, regulador u organización de usuarios puede comprobarlo, que es justo lo que pide el AI Act para sistemas de alto riesgo [13].
- **No cede la marca.** La sección 6 de Apache 2.0 deja fuera los derechos sobre nombres y logotipos. Cualquiera puede modificar el motor de reglas, pero no puede venderlo como “CompanIA” ni presentarlo como certificado. Así separamos la libertad del código de la responsabilidad sobre el robot que llega a las casas.

Licencias que descartamos. La alternativa más seria era GPLv3 [3], que obliga a que cualquier versión modificada siga siendo libre. La descartamos porque su sección 6 exige, en productos de consumo, entregar la información necesaria para instalar software modificado en el aparato. En un robot que levanta peso cerca de personas mayores, eso significaría permitir que cualquiera cambie el sistema de seguridad de una unidad certificada. Además, si nuestro código GPL se enlazara directamente con el SDK propietario del modelo de control, quien redistribuyera el conjunto no podría cumplir la GPL. También descartamos MIT: es igual de permisiva, pero no incluye la protección de patentes.

Riesgo que asumimos. Publicar las reglas de seguridad permite que alguien busque sus huecos. Creemos que compensa: la seguridad que depende de que nadie vea el código es frágil, y es preferible que los fallos los encuentren auditores antes que usuarios, como bien vimos en clase.

1.2 Software propietario: app y plataforma en la nube

La aplicación para familiares y cuidadores, la gestión de avisos y el servicio de mantenimiento remoto quedan cerrados. Es la parte con la que la startup cobra una suscripción, y no toma

decisiones de seguridad sobre el usuario, así que cerrarla no resta transparencia donde importa. Como nuestro código abierto usa una licencia permisiva, combinarlo con software propietario no plantea problemas.

1.3 Librerías y modelos de terceros

Estas licencias no las elegimos nosotros, pero condicionan lo que podemos hacer:

Componente	Licencia	Qué implica para nosotros
ROS 2 (no ROS 1)	Apache 2.0 los paquetes estándar; algunos BSD y alguna herramienta GPL	Compatible con nuestro código. Hay que revisar la licencia paquete a paquete. ROS 1 mezcla BSD, GPL y LGPL, otro motivo para usar ROS 2.
Whisper (voz)	MIT, código y pesos	Podemos afinarlo con voces de personas mayores y usarlo comercialmente.
Gemini Robotics On-Device (control)	Propietaria, mediante SDK y programa de testers [6]	Podemos usarlo y adaptarlo según el contrato con Google, pero no redistribuir el modelo ni lo que derivemos de él.
LLM local (lenguaje y planificación): Gemma 4	Apache 2.0, pesos incluidos [15]	Podemos usarlo, adaptarlo y venderlo dentro del robot conservando el aviso de licencia.

La elección del LLM local sí es nuestra. Elegimos Gemma 4 porque es la primera generación de Gemma publicada bajo Apache 2.0 [15] y está pensada para ejecutarse en local, desde dispositivos pequeños, lo que encaja con procesar los datos dentro del robot. Descartamos Llama [7]: su licencia comunitaria no es software libre, porque impone una política de uso y un límite de usuarios mensuales, y si el producto escalara a muchos usuarios, como analizamos en el sprint 2, quedaríamos en manos de Meta.

Riesgo que asumimos. Con Gemma 4 y Gemini Robotics dependemos de Google en dos componentes. Gracias a Apache 2.0 los pesos de Gemma que ya tenemos no se pueden retirar, pero como alternativa mantenemos modelos de Mistral, también Apache 2.0 y además empresa europea.

Sustitución de modelos. Gemma 4, Whisper y Gemini Robotics son los modelos que usamos hoy, pero no estamos atados a ellos. Si aparece uno que funcione mejor, lo sustituimos siempre que cumpla las mismas condiciones: una licencia que permita uso comercial e integrarlo en el robot (preferiblemente libre, como Apache 2.0 o MIT), ejecución en local dentro del robot y resultados iguales o mejores en las métricas por grupo del anexo de Sesgos y Equidad. Lo que fijamos en este anexo son esos criterios, no los nombres concretos. Esto vale también para el modelo de control: si surgiera uno abierto equivalente a Gemini Robotics, lo preferiríamos para reducir nuestra dependencia de un proveedor.

Un cambio de modelo no se hace sin más. Antes de desplegarlo se repiten las evaluaciones de seguridad y sesgos, se actualiza la model card y este anexo de licencias, y se revisa si el cambio obliga a renovar la evaluación de conformidad del robot como sistema de alto riesgo según el AI Act [13].

2. Modelos propios

CompanIA no entrena modelos desde cero, pero sí adapta varios: el de control a nuestro robot, Whisper a voces de personas mayores y con dificultades del habla, y un detector de caídas. No todos tienen la misma licencia.

2.1 Control y voz: cerrados, con model card pública

El modelo de control y el Whisper afinado no se publican, por dos motivos distintos:

- **Licencia del proveedor.** Lo que derivemos de Gemini Robotics queda sujeto a las condiciones de Google, que no permiten redistribuirlo.
- **Privacidad.** Whisper es MIT, así que legalmente sí podríamos publicar nuestra versión. No lo hacemos porque está entrenada con grabaciones de voluntarios, y un modelo puede memorizar partes de sus datos de entrenamiento. La voz es un dato personal, y tratándose de personas con dificultades del habla puede revelar información de salud.

Para no perder transparencia, sí publicamos la model card de cada modelo: con qué datos se entrenó, cómo funciona en cada grupo de usuarios (las métricas por grupo del anexo de Sesgos y Equidad) y sus limitaciones conocidas. También ofrecemos acceso a los organismos reguladores para auditarlos.

Estos modelos son de lo más valioso de la startup, así que cerrarlos también nos conviene comercialmente. Pero el motivo que lo decide es la privacidad, y lo demuestra que el detector de caídas, que no tiene ese problema, sí lo abrimos.

Licencia que descartamos. Las licencias OpenRAIL-M [8] permiten publicar un modelo prohibiendo ciertos usos. No nos sirven: no resuelven el problema de privacidad, porque una vez publicado los pesos están expuestos igualmente, y no son software libre, porque restringen usos.

2.2 Detector de caídas: Apache 2.0, pesos incluidos

El detector de caídas es la función de seguridad más importante del robot, así que es la que más interesa poder auditar. Lo entrenamos con caídas simuladas por actores y con datasets que permitan uso comercial, sin datos personales de usuarios, y publicamos el modelo completo bajo Apache 2.0. Así cualquiera puede comprobar con qué frecuencia falla y con qué grupos de personas, no solo leerlo en nuestra model card.

La condición es que ningún dato de entrenamiento lo impida. Si lo entrenamos con UR Fall Detection (apartado 3), el modelo queda sujeto a lo que acordemos con sus autores, y si no hay acuerdo comercial no puede usarse en el producto.

3. Datos

No todos los datos se pueden licenciar. Distinguimos cuatro tipos, según el anexo de Componentes de IA y Datos:

- **Datos de los usuarios (rutinas, voz, salud).** No son de CompanIA, sino de cada persona. No llevan licencia: los rige el RGPD, que los trata como categoría especial [11]. Solo se usan con consentimiento explícito y para la finalidad que el usuario haya aceptado, y nunca se publican ni ceden.
- **Grabaciones de voluntarios.** Como base de datos que hemos construido, CompanIA tiene sobre ella el derecho sui generis de la Directiva 96/9/CE [12]. Aun así, al ser datos personales no se publican con una licencia abierta. Además, en personas con dificultades del habla la voz puede revelar información de salud, que es categoría especial. Como mucho se comparten con universidades o asociaciones mediante un acuerdo de uso de datos para investigación, con consentimiento de los voluntarios.

- **Dataset de demostraciones domésticas.** Grabaciones de tareas como recoger objetos o abrir puertas, hechas por actores que ceden sus derechos de imagen por contrato, sin datos de usuarios. Se publican con CC BY 4.0 [10], como aportación a la investigación en robótica asistencial.
- **Datasets públicos.** Debemos respetar las condiciones de cada uno. Esto afecta directamente a UR Fall Detection [14], que proponíamos para entrenar la detección de caídas.

UR Fall Detection se publica bajo CC BY-NC-SA 4.0 y sus autores lo destinan a uso académico no comercial; para fines comerciales piden contactar con ellos. Es decir, su uso comercial está sujeto a un acuerdo previo con los autores (Universidad de Rzeszów). Mientras no exista ese acuerdo, solo lo usamos en la fase de investigación y prototipo, que sí es no comercial, y en paralelo negociamos con ellos una licencia comercial. Si no se llega a un acuerdo, el modelo del producto se entrena con datos propios grabados con voluntarios y actores, y UR Fall solo sirve como referencia para comparar resultados en publicaciones académicas.

Hay un segundo detalle: la cláusula “compartir igual” (SA) obliga a que lo que se derive del dataset se distribuya con la misma licencia. Si llegamos a un acuerdo comercial, sus condiciones sustituyen a esta cláusula para nuestro uso.

4. Documentación: CC BY 4.0

La especificación, los anexos de riesgos y ética, el wiki y los manuales se publican con CC BY 4.0 [10]: cualquiera puede copiarlos, adaptarlos y usarlos, también comercialmente, con la única condición de citarnos. Lo elegimos por tres motivos:

- **Coherencia con el código.** Lo que abrimos lo abrimos sin más condición que la atribución, igual que con Apache 2.0.
- **Reutilización por otros actores.** Asociaciones como CEOMA o CERMI, administraciones o residencias podrían incorporar partes de nuestras guías a sus propios materiales sin cambiar la licencia de estos.
- **Encaje con la startup.** Distribuidores y socios pueden adaptar los manuales a su versión del producto.

Licencias que descartamos. CC BY-SA 4.0 [9] habría impedido que alguien cerrara versiones mejoradas de nuestra guía ética, pero obligaría a quien reutilice partes a publicar sus materiales con la misma licencia, y muchas organizaciones no podrían o no querrían. Las variantes NC (no comercial) y ND (sin obras derivadas) contradicen el objetivo de que otros reutilicen la documentación y no encajan con una empresa.

Los ejemplos de código que aparezcan dentro de la documentación van bajo Apache 2.0, no Creative Commons, porque Creative Commons desaconseja usar sus licencias para software: no cubren patentes ni están pensadas para código.

5. Hardware: propietario

Existen licencias de hardware libre, como CERN-OHL, que permitirían publicar el diseño del robot. Las descartamos para el robot completo: su seguridad física depende de que el diseño certificado no se modifique, y si se publicara podrían aparecer copias sin certificar usadas con personas vulnerables. Sí podemos abrir en el futuro accesorios que no afectan a la seguridad.

6. Incompatibilidades entre licencias

Ninguna de estas incompatibilidades impide el proyecto. Las identificamos para justificar por qué hemos elegido estas licencias y qué limitaciones aceptamos.

- **Apache 2.0 y GPL.** El código Apache 2.0 se puede incluir en un proyecto GPLv3, pero no al revés, y es incompatible con GPLv2 por la cláusula de patentes [2]. Por eso hay que auditar la licencia de cada paquete de ROS 2 que usemos y no distribuir en el producto nada GPL enlazado con nuestro código. Las herramientas GPL que solo se usen durante el desarrollo y no vayan en el robot no nos afectan.
- **Código libre y SDK propietario.** Con Apache 2.0 no hay problema. Con GPL, enlazar directamente con el SDK de Gemini lo habría generado. Además, los nodos de ROS se comunican mediante mensajes entre programas separados, lo que reduce el riesgo de que se consideren una sola obra.
- **Gemini y nuestros modelos.** Su licencia impide publicar el modelo de control afinado, aunque quisiéramos. Tampoco podemos presentarlo como compatible con Apache 2.0: es un componente propietario que usamos bajo contrato.
- **UR Fall y uso comercial.** Su licencia CC BY-NC-SA 4.0 no permite usarlo en un producto comercial sin permiso de los autores. Es la única incompatibilidad que nos obliga a actuar: negociar una licencia comercial o sustituirlo por datos propios.
- **Código y documentación.** Tienen licencias distintas, así que no se deben copiar fragmentos de código en la documentación ni de la documentación al código sin indicar la licencia de cada parte.

7. Aplicación en nuestro repositorio

En la raíz del repositorio hemos añadido un archivo LICENSE con Apache 2.0, que se aplica al código que contiene, empezando por el mockup interactivo del sprint 2, y a cualquiera que añadamos. Es el único archivo de licencias del repositorio: el desglose de la licencia de cada componente, propio y de terceros, está en este anexo (tabla Resumen y apartado 1.3).

Referencias

[1] Apache Software Foundation (2004). Apache License, Version 2.0. https://www.apache.org/licenses/LICENSE-2.0. Licencia elegida para nuestro código. Usamos en concreto la sección 3 (concesión de patentes) y la sección 6 (no cede derechos de marca).

[2] Apache Software Foundation. Apache License v2.0 and GPL Compatibility. https://www.apache.org/licenses/GPL-compatibility.html. Explica por qué Apache 2.0 es compatible con GPLv3 en un solo sentido e incompatible con GPLv2. Base del apartado de incompatibilidades.

[3] Free Software Foundation (2007). GNU General Public License v3.0. https://www.gnu.org/licenses/gpl-3.0.html. Licencia que comparamos y descartamos. La sección 6 obliga a entregar la “Installation Information” cuando el software se distribuye en un “User Product”, y es el motivo principal del descarte.

[4] ROS 2 (2023). rcl_action: Quality Declaration, sección 3.iii (License). https://github.com/ros2/rcl/blob/6f9fe6d43a306b57eeca6495c10cf8c2906472f9/rcl_action/QUALITY_DECLARATION.md. Ejemplo de paquete del núcleo de ROS 2 que declara su licencia como Apache 2.0. Las declaraciones de calidad de los paquetes del núcleo incluyen esta misma sección, lo que justifica que nuestro código use la misma licencia.

[5] OpenAI (2022). Whisper (repositorio y licencia MIT). https://github.com/openai/whisper. Modelo de reconocimiento de voz que adaptamos. Su licencia MIT cubre código y pesos, por lo que podemos afinarlo.

[6] Google DeepMind (2025). Gemini Robotics On-Device brings AI to local robotic devices. https://deepmind.google/blog/gemini-robotics-on-device-brings-ai-to-local-robotic-devices/. Muestra que el modelo de control se ofrece mediante un programa de testers y un SDK, no como modelo abierto. Por eso nuestro componente de control es cerrado.

[7] Meta (2024). Llama 3.1 Community License Agreement. https://www.llama.com/llama3_1/license/. Ejemplo de licencia de modelo "abierto" que no es software libre: la sección 1.b.iv obliga a cumplir su política de uso aceptable y la sección 2 exige una licencia aparte por encima de 700 millones de usuarios activos mensuales. La usamos como contraejemplo al elegir el LLM local.

[8] BigScience (2022). BigScience OpenRAIL-M License. https://www.licenses.ai/blog/2022/8/26/bigscience-open-rail-m-license. Licencia de modelo con restricciones de uso que comparamos y descartamos para nuestros modelos afinados.

[9] Creative Commons (2013). Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0). https://creativecommons.org/licenses/by-sa/4.0/deed.es. Licencia comparada y descartada para la documentación.

[10] Creative Commons (2013). Atribución 4.0 Internacional (CC BY 4.0). https://creativecommons.org/licenses/by/4.0/deed.es. Licencia elegida para la documentación y para el dataset de demostraciones domésticas.

[11] Unión Europea (2016). Reglamento (UE) 2016/679 (RGPD), art. 9. https://eur-lex.europa.eu/legal-content/ES/TXT/HTML/?uri=CELEX:32016R0679#art_9. El art. 9.1 incluye los datos de salud entre las categorías especiales, y el art. 4.15 define dato de salud. Explica por qué los datos de usuario no se pueden "licenciar".

[12] Unión Europea (1996). Directiva 96/9/CE sobre la protección jurídica de las bases de datos. https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:31996L0009. El art. 7 regula el derecho sui generis que tiene CompanIA sobre las bases de datos que recopila con voluntarios.

[13] Unión Europea (2024). Reglamento (UE) 2024/1689 (AI Act). https://eur-lex.europa.eu/legal-content/ES/TXT/PDF/?uri=OJ:L_202401689. Requisitos para IA de alto riesgo: registros (art. 12), transparencia (art. 13) y supervisión humana (art. 14), que refuerzan la decisión de abrir el motor de decisión; y nueva evaluación de conformidad ante una modificación sustancial (art. 43.4), aplicable al cambiar de modelo.

[14] Kwolek, B. y Kepski, M. (2014). UR Fall Detection Dataset. Universidad de Rzeszów. https://fenix.ur.edu.pl/~mkepski/ds/uf.html. Dataset público que citamos en el anexo de Componentes. Licencia CC BY-NC-SA 4.0, solo uso académico no comercial; el uso comercial requiere contactar con los autores.

[15] Google Open Source Blog (2026). Gemma 4: Expanding the Gemmaverse with Apache 2.0. https://opensource.googleblog.com/2026/03/gemma-4-expanding-the-gemmaverse-with-apache-20.html. Afirma que los modelos Gemma 4 son los primeros de la familia publicados bajo Apache 2.0, aprobada por la OSI, y que permiten ejecución local. Justifica la elección del LLM local.

AI Logs

https://claude.ai/share/cc7832b1-d24d-448a-a79e-35c24008d2fc - Lluvia de ideas y ayuda para saber abordar la tarea

También se utilizó IA para ayudar en la redacción y en la búsqueda de referencias, siempre siendo verificadas por nosotros.

Crítica a la IA. La IA afirmó que la voz es un dato biométrico sin matizarlo. Al revisar el RGPD vimos que solo lo es cuando se trata para identificar a una persona (art. 4.14), y CompanIA

no usa la voz para eso. Lo que la convierte en categoría especial en nuestro caso es que, en personas con dificultades del habla, puede revelar información de salud (art. 4.15). Corregimos el argumento en el apartado de datos